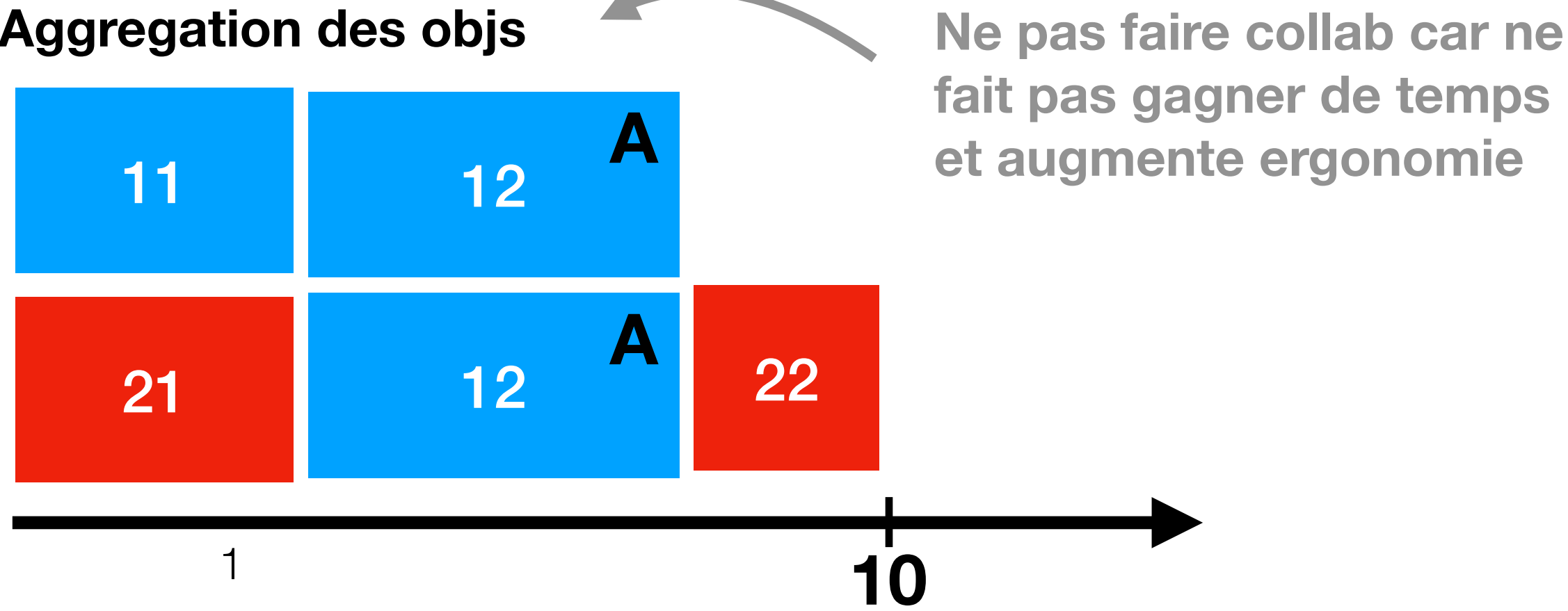
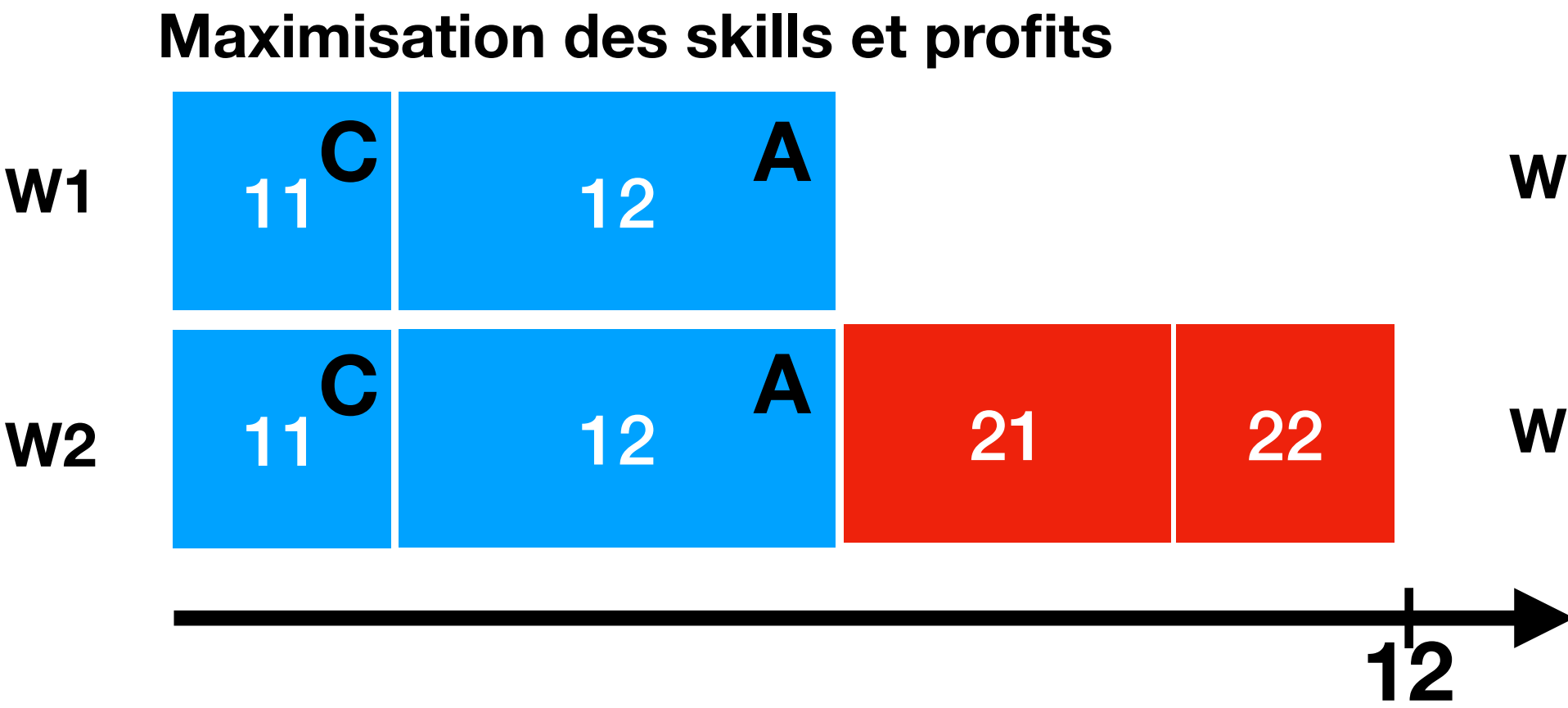
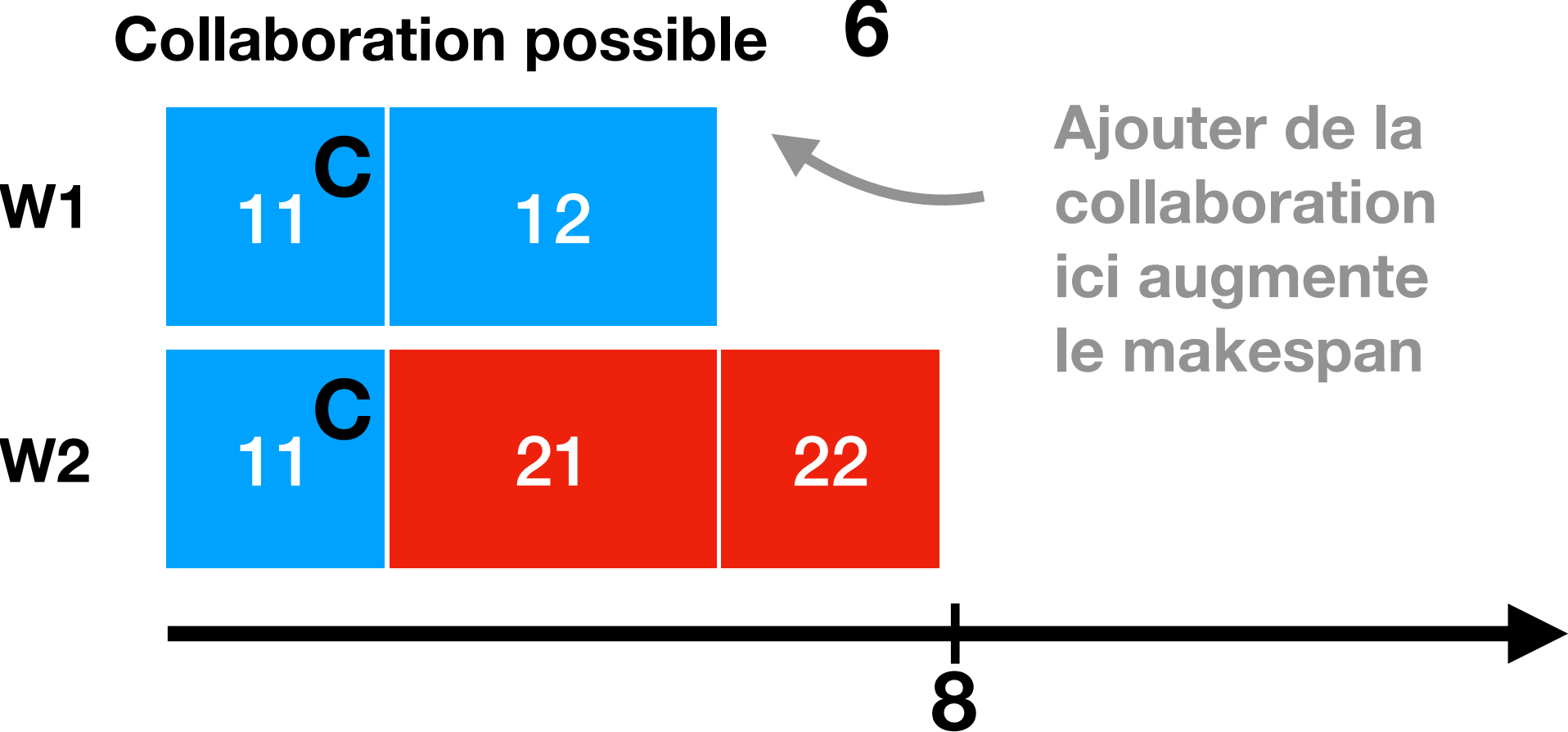
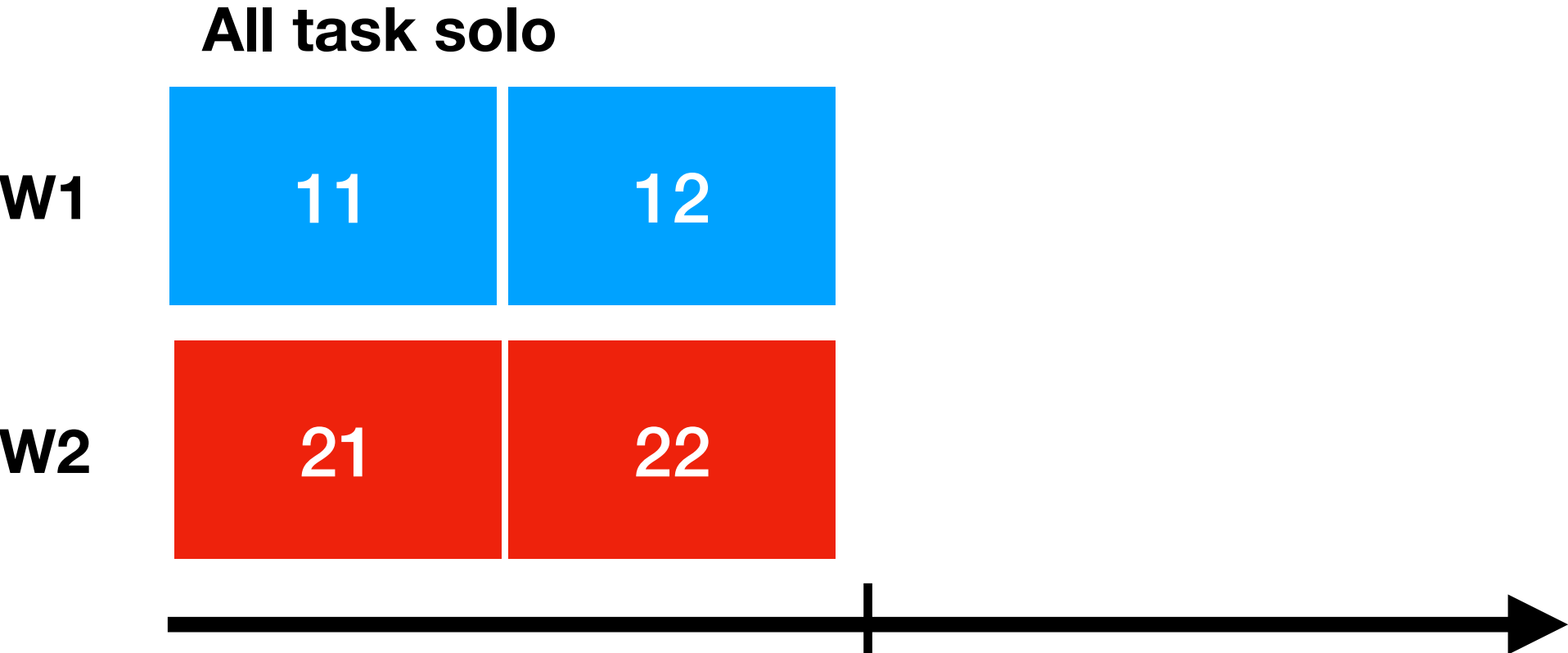


Instance de test fait mains

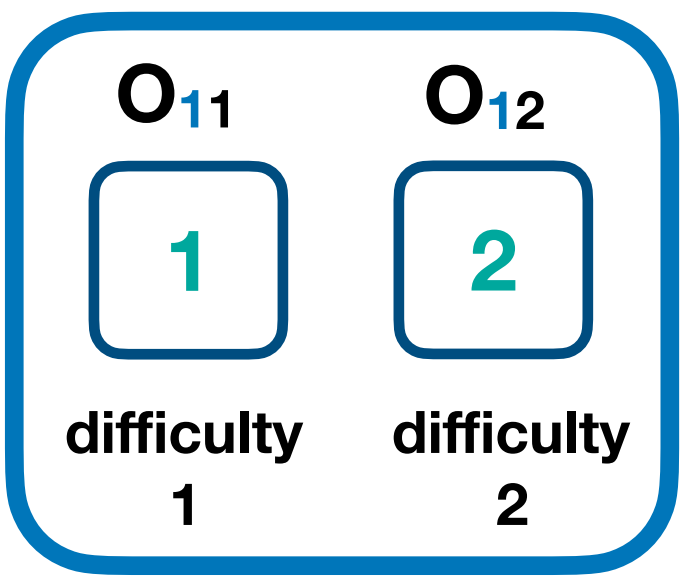


4 opérations de temps identique

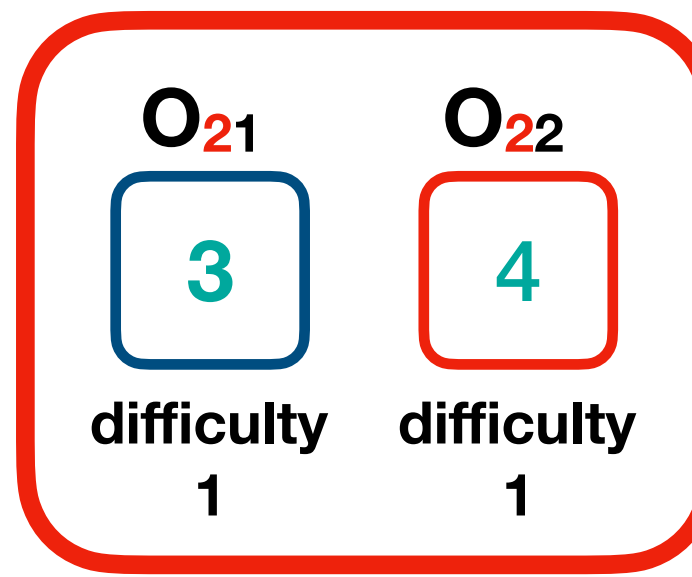
Solo	App	Collab
3	4	2

W1	W2
2	1

J₁

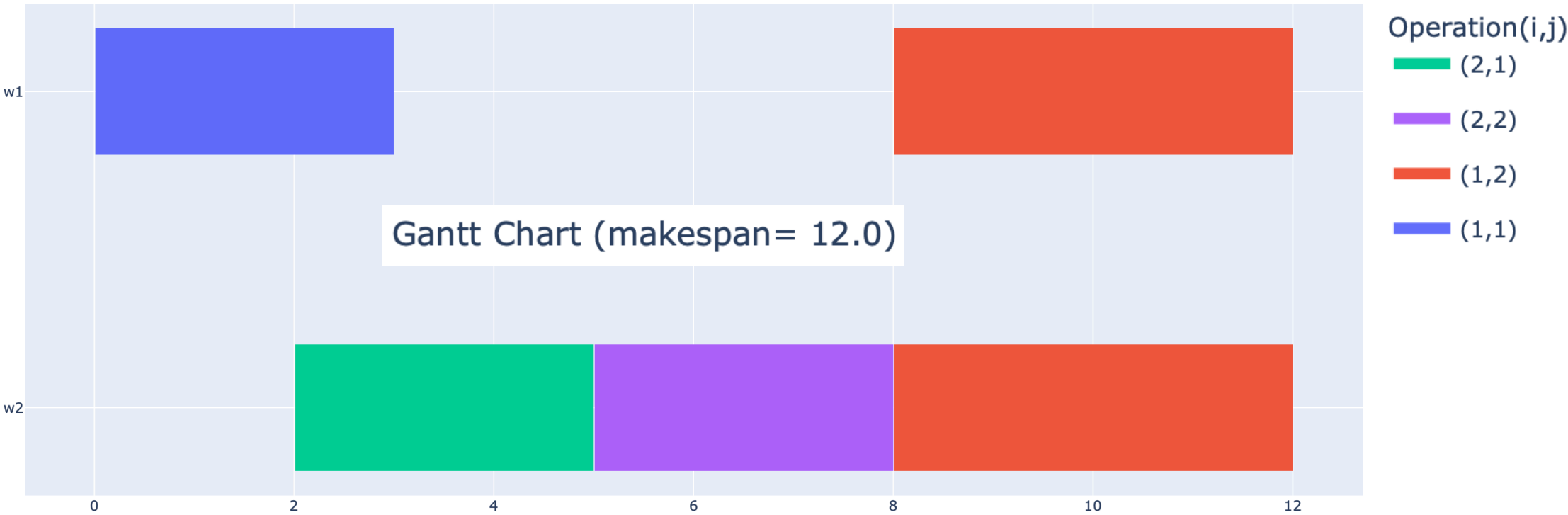


J₂

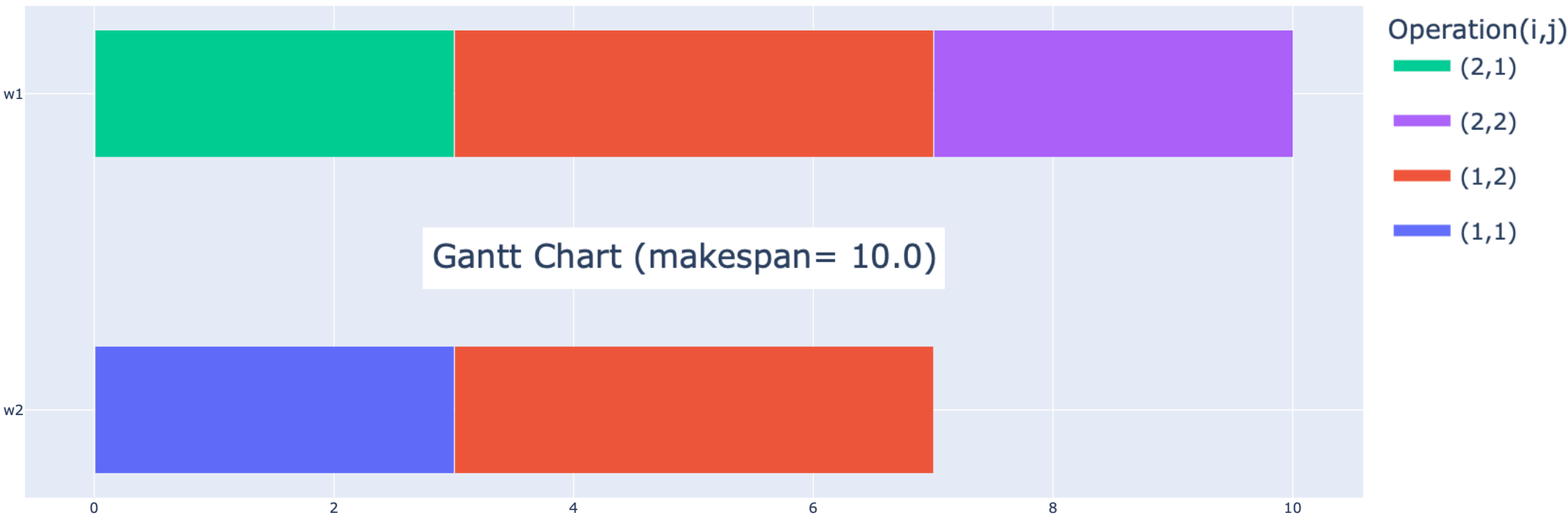


Modification de la solution quand C_max (makespan) est pris en compte dans la fonction objectif

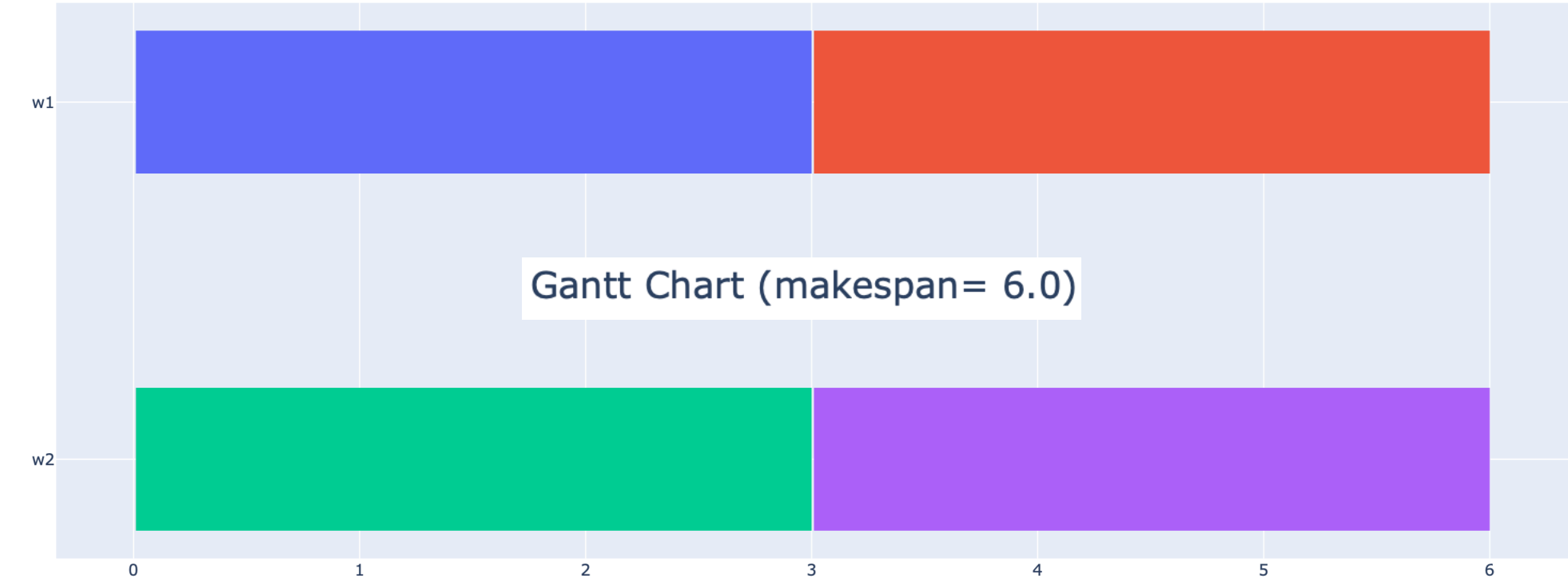
$$\sum (profit - penalty_time) \cdot \alpha_1 + (level_depart - level_arrive) \cdot \alpha_2 + (cognitive_load) \cdot \alpha_3$$



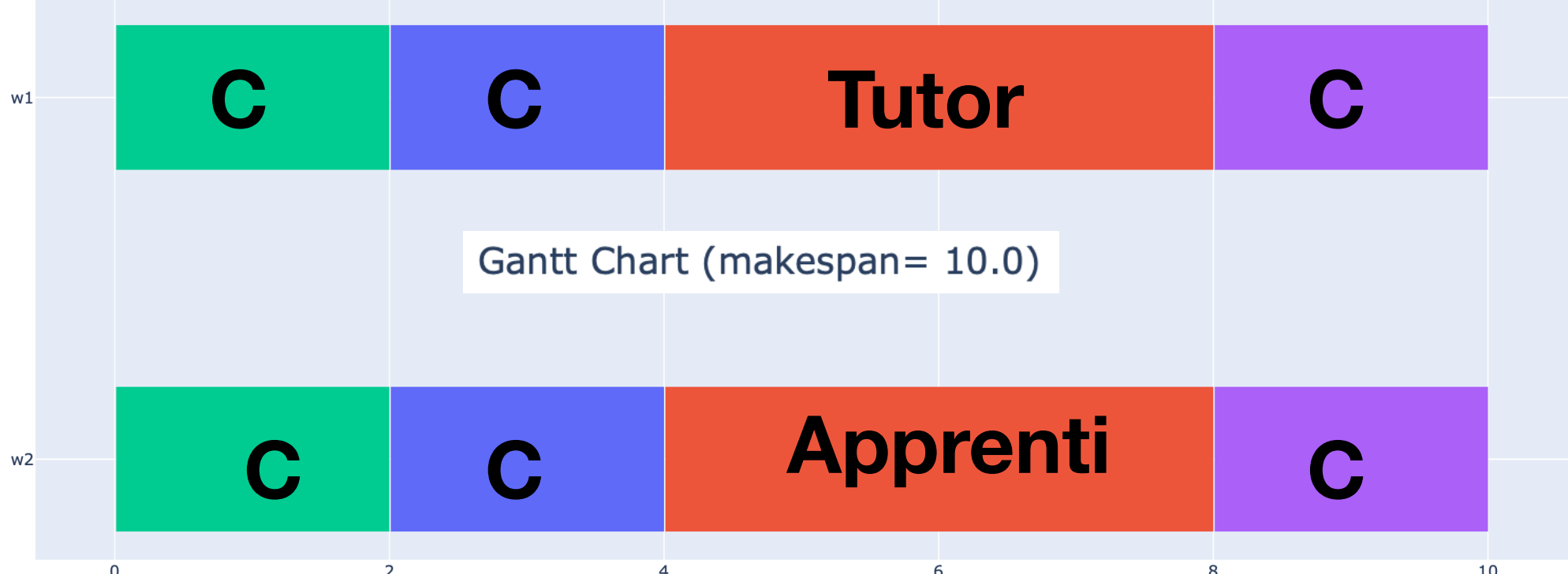
$$\sum (profit - penalty_time) \cdot \alpha_1 + (level_depart - level_arrive) \cdot \alpha_2 + (cognitive_load) \cdot \alpha_3 - C_{max}$$



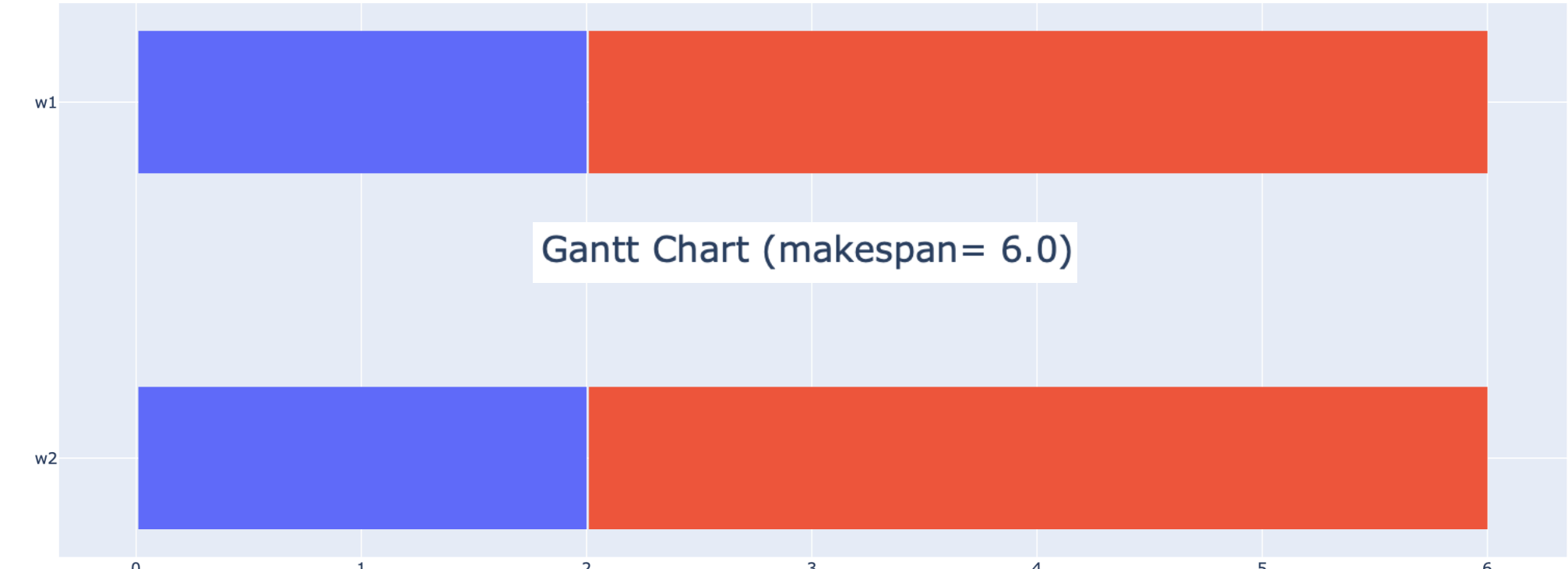
All task solo



No task solo

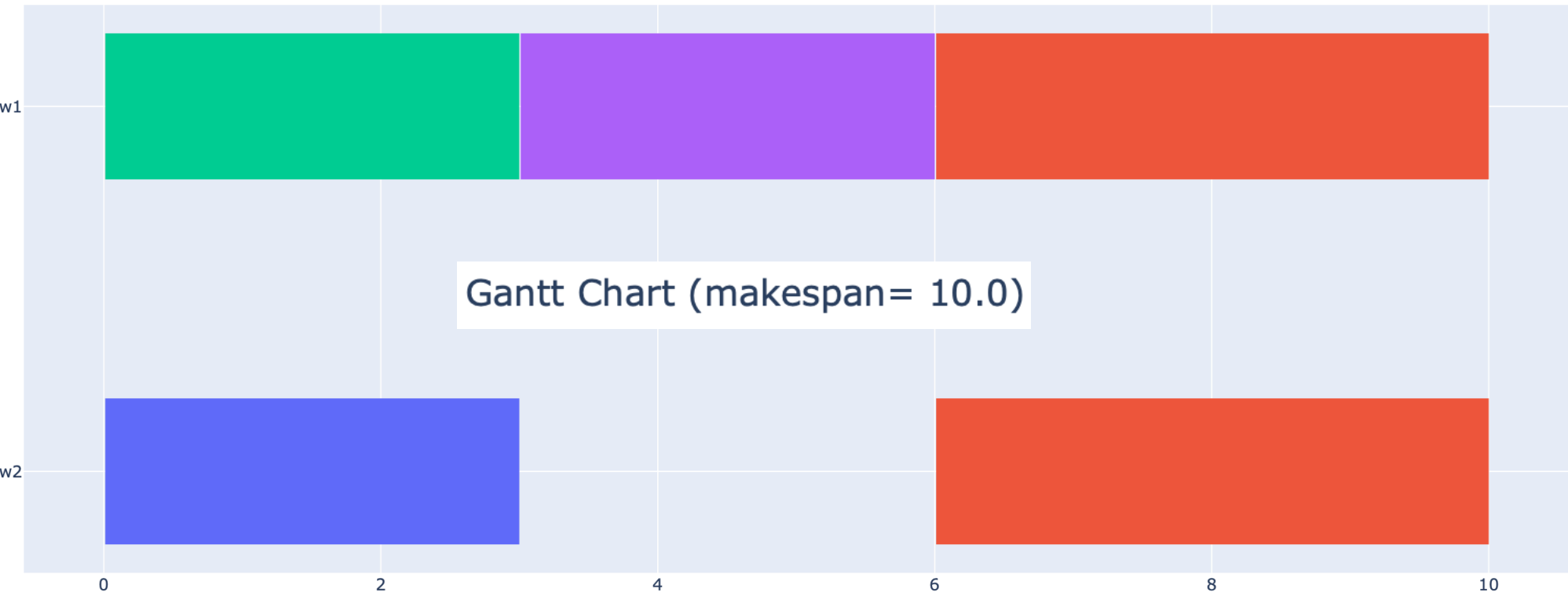


Max skills



	Task	Start	Finish	Processing time	Operation	Job	mode	Level_w	Difficulty_task
0	w1	0.01	2.0	2.0	(2,1)	J2	collaboratively	2.0	1
1	w1	2.01	4.0	2.0	(1,1)	J1	collaboratively	2.0	1
2	w1	4.01	8.0	4.0	(1,2)	J1	learning (tutor)	2.0	2
3	w1	8.01	10.0	2.0	(2,2)	J2	collaboratively	2.0	1
4	w2	0.01	2.0	2.0	(2,1)	J2	collaboratively	1.0	1
5	w2	2.01	4.0	2.0	(1,1)	J1	collaboratively	1.0	1
6	w2	4.01	8.0	4.0	(1,2)	J1	learning (apprentice)	1.0	2
7	w2	8.01	10.0	2.0	(2,2)	J2	collaboratively	1.0	1

Lex (skill, profit, ergonomie)



3. Définition de la fonction objectif

K ens. des workers

M ens. des métiers

$$\max \sum_i \text{resal_price}[i] \cdot \text{job_done_before_limit}[i] \quad (1)$$

$$\max \sum_{k \in W, m \in M} \text{level_final}[k, m] - \text{level_depart}[k, m] \quad (2)$$

$$\min \sum_{k \in W, m \in M} \text{cognitive_load}[k, m] \quad (3)$$

Aggrégation des critères

$$\sum (1) \cdot \alpha_1 + (2) \cdot \alpha_2 + (3) \cdot \alpha_3 - C_{\max}$$

Si on autorise soft constraint level

$$\min \sum_{i \in J, j \in J_i} \text{penalty_level}[i, j] \quad (4)$$

Pour limiter les opérations fait sans levels

$$\min \sum_i \text{penalty_makespan_job}[i] \quad (5)$$

En maximisant (1) sa force à minimiser (5) mais ne force pas à réduire la variable pour un job donnée si il dépasse le temps limite. Donc affectation apres la limite de temps négligé

Dans mon modèle MILP **skills** et **cognitive_load** sont calculé sur la base de la variable d'affectation x_{ijk} et cette variable depend de **job_done** et non pas **job_done_before_limit** cela permet de visualiser les affectations même pour les job fini plus tard, mais **skills** et **cognitive_load** les ajoutent dans leurs résultats

Contrainte du modèle

Si $job_done = 0$ alors aucune tâche du job est affecté

$$x_{ijk} \leq job_done_i \quad \forall i \in J, j \in J_i, k \in W \quad (26)$$

$$l_{km} \leq L_{km} + \left(\sum_{\substack{i \in J, \\ j \in J_i, \\ O_{ij} \in m, \\ L_{km} < D_{ij}}} x_{ijk} \right) \cdot COEFF_L \quad \forall k \in W, m \in M$$

Pour avoir *skills* et *cognitive_load* calculé sur la base des opérations fait avant le temps limite seulement

1- Soit on modifie la contrainte (26) en $x_{ijk} \leq job_done_before_limit[i]$ mais plus de visualisation d'affectation après le temps limite.

2- Soit on garde la contrainte (26) mais on ajoute une variable x'_{ijk} qui sert pour l'affectation des opérations fait avant le temps limite.